МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №5

з курсу: „ Інтернет речей та вбудовані системи”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

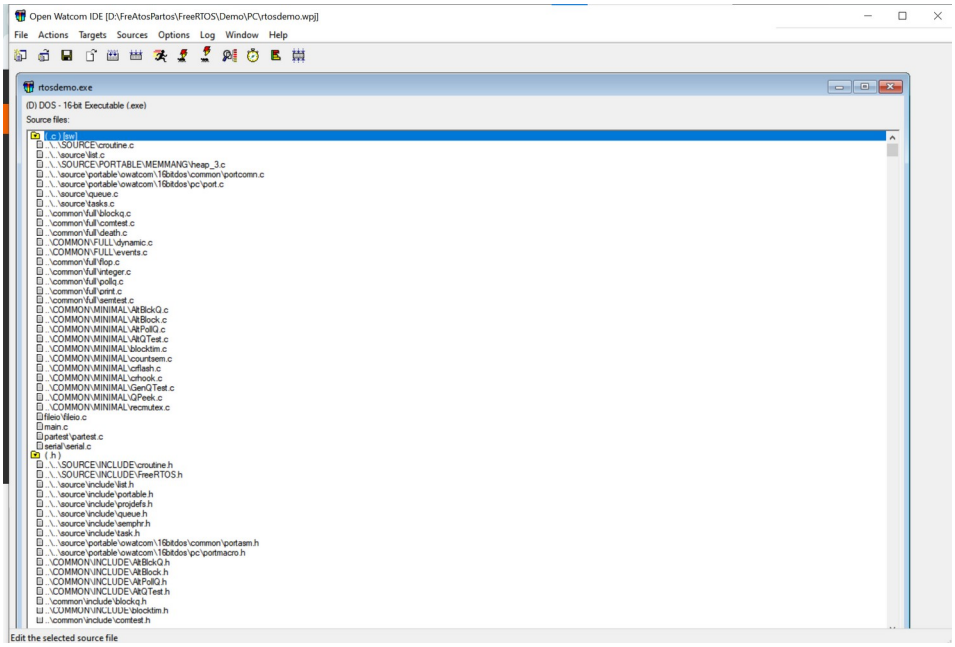
КИЇВ
2025

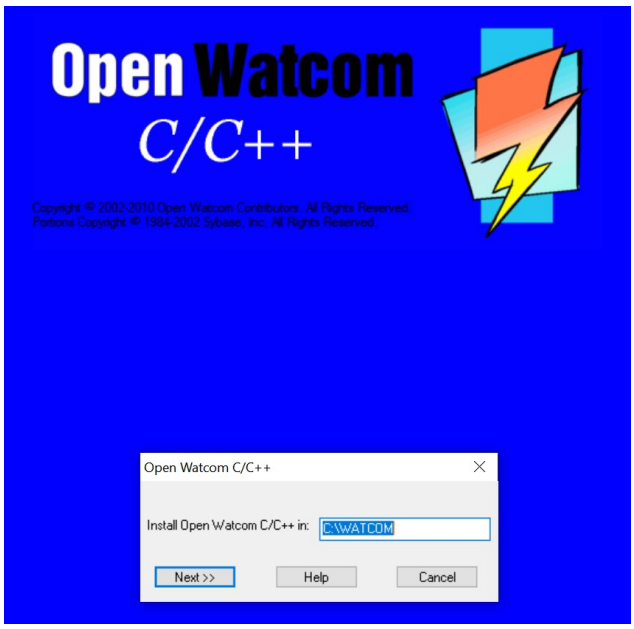
Мета роботи: вивчити основні принципи роботи операційної системи (ОС) FreeRTOS в ПЗ Open Watcom.

Хід роботи

1. FreeRTOS







2. Створити проект з однією періодичною задачею, що запускається кожні 300 мс.

У даному пункті створюється проект з однією задачею операційної системи реального часу FreeRTOS, яка виконується періодично з фіксованим інтервалом 300 мілісекунд (демонстрація принципу періодичного виконання задач та використання механізму блокування задачі на певний час без зайвого навантаження на процесор.)

```
void vPeriodicTask300ms(void* pvParameters)
{
   (void)pvParameters;

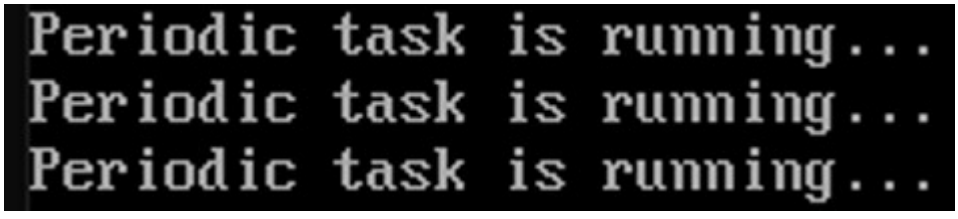
   for (;;)
   {
      vTaskSuspendAll();
      printf("Periodic task executed every 300 ms\r\n");
      xTaskResumeAll();
      vTaskDelay(pdMS_TO_TICKS(300));
   }
}

int main(void)
{
   xTaskCreate(
      vPeriodicTask300ms,
      "Periodic300",
      1000,
      NULL,
      tskIDLE_PRIORITY + 1,
      NULL
   );

   vTaskStartScheduler();

   for (;;);
}
```

У програмі створюється одна задача vPeriodicTask300ms, яка реалізована у вигляді нескінченного циклу. Всередині циклу виконується виведення повідомлення в консоль, після чого задача переводиться у стан блокування на 300 мілісекунд за допомогою функції vTaskDelay. Для коректної роботи використовується макрос pdMS_TO_TICKS, який перетворює мілісекунди у системні тіки. Виклик vTaskSuspendAll та xTaskResumeAll тимчасово зупиняє планувальник, що запобігає пошкодженню виводу під час виконання функції printf. У функції main створюється одна задача з пріоритетом, вищим за задачу простою, після чого запускається планувальник FreeRTOS.



3. Створити проект з двома періодичними задачами з різними пріоритетами, які запускаються кожні 500 і 750 мс. Проаналізувати процес витіснення низькопріоритетної задачі високпріоритетною з побудовою діаграми послідовністю їх виконання.

Проект з двома періодичними задачами FreeRTOS, які мають різні пріоритети та різні періоди виконання 500 та 750 мілісекунд (дослідження механізму витіснення, за якого задача з вищим пріоритетом перериває виконання задачі з нижчим пріоритетом у момент переходу в стан готовності)

```
void vHighPriorityTask(void* pvParameters)
{
   (void)pvParameters;

   for (;;)
   {
      vTaskSuspendAll();
      printf("Task 1 (High Priority) is running\r\n");
      xTaskResumeAll();

      vTaskDelay(pdMS_TO_TICKS(500));
   }
}

void vLowPriorityTask(void* pvParameters)
{
   (void)pvParameters;

   for (;;)
   {
      vTaskSuspendAll();
      printf("Task 2 (Low Priority) is running\r\n");
      xTaskResumeAll();

      vTaskDelay(pdMS_TO_TICKS(750));
   }
}

int main(void)
{
   xTaskCreate(
      vHighPriorityTask,
      "HighTask",
      1000,
      NULL,
      3,        // високий пріоритет
      NULL
   );

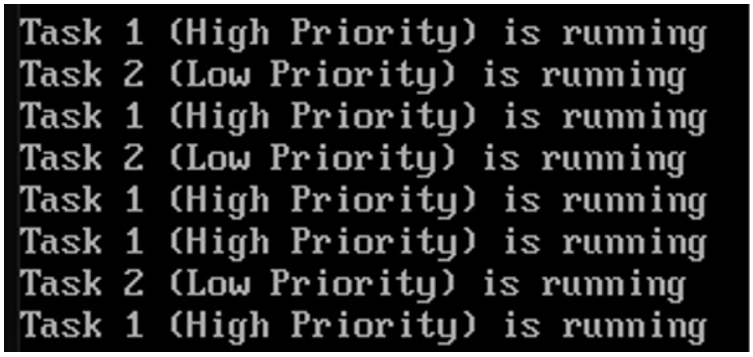
   xTaskCreate(
      vLowPriorityTask,
      "LowTask",
      1000,
      NULL,
      1,        // низький пріоритет
      NULL
   );

   vTaskStartScheduler();
```

```
    for (;;);
}
```

У програмі створюються дві задачі з різними пріоритетами. Задача vHighPriorityTask має вищий пріоритет і виконується кожні 500 мілісекунд, тоді як задача vLowPriorityTask має нижчий пріоритет і виконується кожні 750 мілісекунд. Обидві задачі реалізовані у вигляді нескінченних циклів з використанням vTaskDelay, що переводить їх у стан блокування на заданий час. Планувальник FreeRTOS працює у витісняючому режимі, тому у випадку, коли високопріоритетна задача переходить у стан готовності, вона негайно перериває виконання низькопріоритетної задачі та отримує керування процесором.

Після запуску планувальника першою починає виконуватися задача з вищим пріоритетом. Після завершення виведення повідомлення вона переходить у стан блокування на 500 мілісекунд. У цей момент процесор передається задачі з нижчим пріоритетом. Якщо під час виконання низькопріоритетної задачі завершується затримка високопріоритетної задачі, вона переходить у стан готовності та миттєво витісняє низькопріоритетну задачу. Таким чином, низькопріоритетна задача виконується лише у проміжках часу, коли високопріоритетна задача заблокована.



4. Створити чотири задачі. Три задачі, які мають різний пріоритет записують дані в чергу. Одна задача з найвищим пріоритетом читає дані з черги.

Створюються три задачі-письменники з різними пріоритетами, які передають числові дані у спільну чергу, та одна задача-читач з найвищим пріоритетом, яка приймає та обробляє ці дані. Метою завдання є демонстрація потокобезпечної передачі даних між задачами та аналіз впливу пріоритетів на порядок їх виконання.

У програмі спочатку створюється черга, розрахована на зберігання чотирьох елементів типу int. Далі ініціалізуються три задачі-письменники, кожна з яких отримує власне числове значення через параметр та має різний пріоритет. Задачі-письменники періодично намагаються передати дані в чергу. Якщо черга переповнена, задача виводить повідомлення про очікування та

блокується до моменту звільнення місця. Задача-читач створюється з найвищим пріоритетом і перебуває у заблокованому стані, очікуючи появи даних у черзі. Після отримання елемента вона виводить його значення та знову переходить у стан затримки.

```
QueueHandle_t xQueue;

static void vWriterTask(void* pvParameters)
{
   int valueToSend = (int)pvParameters;
   const TickType_t xDelay = pdMS_TO_TICKS(2000);

   for (;;)
   {
      vTaskSuspendAll();
      printf("Writer task sending: %d\r\n", valueToSend);
      xTaskResumeAll();

      if (xQueueSend(xQueue, &valueToSend, 0) != pdPASS)
      {
         vTaskSuspendAll();
         printf("Writer with value %d: Queue is full, WAITING...\r\n", valueToSend);
         xTaskResumeAll();

         xQueueSend(xQueue, &valueToSend, portMAX_DELAY);
      }

      vTaskDelay(xDelay);
   }
}

static void vReaderTask(void* pvParameters)
{
   int receivedValue;
   const TickType_t xReaderDelay = pdMS_TO_TICKS(500);

   for (;;)
   {
      xQueueReceive(xQueue, &receivedValue, portMAX_DELAY);

      vTaskSuspendAll();
      printf("Reader task received: %d\r\n", receivedValue);
      xTaskResumeAll();

      vTaskDelay(xReaderDelay);
   }
}

int main(void)
{
   prvSetupHardware();

   xQueue = xQueueCreate(4, sizeof(int));

   if (xQueue != NULL)
   {
      xTaskCreate(
         vWriterTask,
         "Writer 1 (min)",
         configMINIMAL_STACK_SIZE,
         (void*)1,
```

```
            tskIDLE_PRIORITY + 1,
            NULL
        );

        xTaskCreate(
            vWriterTask,
            "Writer 2 (avg)",
            configMINIMAL_STACK_SIZE,
            (void*)5,
            tskIDLE_PRIORITY + 2,
            NULL
        );

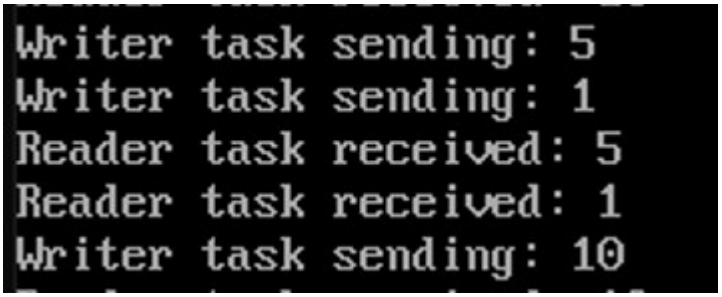
        xTaskCreate(
            vWriterTask,
            "Writer 3 (max)",
            configMINIMAL_STACK_SIZE,
            (void*)10,
            tskIDLE_PRIORITY + 3,
            NULL
        );

        xTaskCreate(
            vReaderTask,
            "Reader",
            configMINIMAL_STACK_SIZE,
            NULL,
            tskIDLE_PRIORITY + 4,
            NULL
        );

        vTaskStartScheduler();
    }

    for (;;);
}

static void prvSetupHardware(void)
{
}
```



Під час виконання програми у консолі відображаються повідомлення про відправлення даних задачами-письменниками та повідомлення про прийом даних задачею-читачем. У разі переповнення черги з’являється повідомлення про очікування. Вивід підтверджує, що задача-читач негайно обробляє дані після їх появи в черзі та витісняє задачі-письменники завдяки вищому пріоритету.

5. Створити проект з трьох задач, що демонструє доступ до захищеного ресурсу через задачу-вахтера. Захищений ресурс – виведення даних на екран. Дві задачі надсилають повідомлення через чергу третій задачі-вахтеру. Третя задача друкує повідомлення на екрані.

У пункті створюється проект з трьох задач, який демонструє організацію безпечного доступу до захищеного ресурсу за допомогою задачі-вахтера. Захищеним ресурсом є виведення інформації на екран. Дві задачі-відправники не мають прямого доступу до виводу, а передають текстові повідомлення через чергу спеціальній задачі-вахтеру, яка виконує друк у консоль.

```
static const char* pcMessagesToSend[] =
{
   "Message from Sender 1 *\r\n",
   "Message from Sender 2 -\r\n"
};

QueueHandle_t xPrintQueue;

int main(void)
{
   prvSetupHardware();

   xPrintQueue = xQueueCreate(5, sizeof(char*));

   if (xPrintQueue != NULL)
   {
      xTaskCreate(
         vSenderTask,
         "Sender 1",
         configMINIMAL_STACK_SIZE,
         (void*)pcMessagesToSend[0],
         tskIDLE_PRIORITY + 1,
         NULL
      );

      xTaskCreate(
         vSenderTask,
         "Sender 2",
         configMINIMAL_STACK_SIZE,
         (void*)pcMessagesToSend[1],
         tskIDLE_PRIORITY + 1,
         NULL
      );

      xTaskCreate(
         vGatekeeperTask,
         "Gatekeeper",
         configMINIMAL_STACK_SIZE,
         NULL,
         tskIDLE_PRIORITY,
         NULL
      );

      vTaskStartScheduler();
   }
```

```
    for (;;);
    return 0;
}

static void vSenderTask(void* pvParameters)
{
    char* pcMessageToSend;
    TickType_t xDelayTime;

    pcMessageToSend = (char*)pvParameters;

    if (pcMessageToSend == pcMessagesToSend[0])
    {
        xDelayTime = pdMS_TO_TICKS(1000);
    }
    else
    {
        xDelayTime = pdMS_TO_TICKS(1500);
    }

    for (;;)
    {
        xQueueSend(xPrintQueue, &pcMessageToSend, portMAX_DELAY);
        vTaskDelay(xDelayTime);
    }
}

static void vGatekeeperTask(void* pvParameters)
{
    char* pcReceivedMessage;

    for (;;)
    {
        xQueueReceive(xPrintQueue, &pcReceivedMessage, portMAX_DELAY);
        printf("%s", pcReceivedMessage);
        fflush(stdout);
    }
}

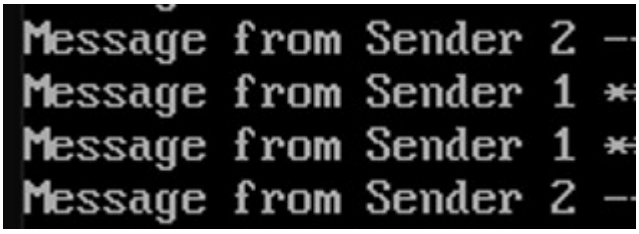
static void prvSetupHardware(void)
{
}
```

У даному завданні реалізовано доступ до захищеного ресурсу з використанням задачі-вахтера. Захищеним ресурсом є виведення текстової інформації на екран, яке не є потокобезпечним при одночасному доступі з кількох задач. Для демонстрації цього підходу було створено три задачі операційної системи FreeRTOS.

Дві задачі-відправники періодично надсилають текстові повідомлення до спільної черги. Кожна з цих задач має власний інтервал затримки, що дозволяє спостерігати асинхронне надсилання повідомлень. Задачі-відправники не виконують виведення даних напряму, а лише передають повідомлення задачі-вахтеру.

Третя задача виконує роль вахтера і є єдиною задачею, яка має прямий доступ до виведення інформації на екран. Вона постійно очікує появи повідомлень у черзі, після чого послідовно зчитує їх та виводить у консоль.

Оскільки лише одна задача виконує друк, повідомлення не змішуються та відображаються коректно.



Під час виконання програми у консолі з’являються повідомлення від двох задач-відправників, які виводяться почергово задачею-вахтером. Вивід є впорядкованим і не містить пошкоджених або змішаних рядків, що підтверджує коректну реалізацію доступу до захищеного ресурсу через задачу-вахтера.

Висновок.

Під час виконання даної лабораторної роботи було детально вивчено принципи роботи операційної системи реального часу FreeRTOS та її основні механізми керування задачами. У ході роботи було реалізовано кілька прикладів багатозадачних програм, що дозволило на практиці дослідити особливості планування задач, використання пріоритетів та організацію періодичного виконання коду.

У процесі виконання завдань було продемонстровано роботу витісняючого планувальника FreeRTOS. На прикладі задач з різними пріоритетами було підтверджено, що система завжди надає процесорний час задачі з вищим пріоритетом, тоді як задачі з нижчим пріоритетом виконуються лише у моменти, коли високопріоритетні задачі перебувають у стані блокування. Це дозволяє забезпечити детерміновану поведінку системи та своєчасне виконання критично важливих операцій.

Особливу увагу було приділено міжзадачній взаємодії з використанням черг. Було реалізовано обмін даними між кількома задачами-письменниками та задачею-читачем, що продемонструвало надійність та потокобезпечність механізму черг у FreeRTOS. Задача-читач з вищим пріоритетом ефективно обробляла дані одразу після їх появи, не створюючи зайвого навантаження на процесор.

Також у лабораторній роботі було реалізовано патерн доступу до захищеного ресурсу через задачу-вахтера. Даний підхід дозволив організувати безпечне використання спільного ресурсу без застосування м’ютексів або критичних секцій у кожній задачі. Централізація операцій виведення інформації в одній задачі забезпечила коректний та впорядкований вивід даних.

Отримані результати підтверджують, що FreeRTOS є ефективною та гнучкою операційною системою реального часу для вбудованих систем, яка забезпечує зручні механізми керування задачами, синхронізації та обміну

даними. Практичне виконання лабораторної роботи дозволило закріпити теоретичні знання та сформувати навички розробки багатозадачних програм для систем реального часу.

Контрольні питання

3.1. Опишіть випадки, коли доцільно використовувати ОС реального часу.

Операційні системи реального часу доцільно використовувати у тих системах, де коректність роботи залежить не лише від правильності обчислень, а й від часу отримання результату. У таких системах важливо, щоб реакція на події відбувалася в гарантований і передбачуваний проміжок часу. До них належать системи керування технологічними процесами, робототехніка, автомобільні електронні системи, медичне обладнання та авіаційні комплекси. Застосування ОС реального часу дозволяє забезпечити детерміновану поведінку системи та своєчасне виконання критично важливих задач.

3.2. Охарактеризуйте принципи функціонування ОС FreeRTOS.

FreeRTOS є компактною операційною системою реального часу, призначеною для використання у вбудованих системах та мікроконтролерах. Її робота базується на принципі багатозадачності, коли кілька задач виконуються псевдопаралельно на одному процесорному ядрі. Центральним елементом системи є витісняючий планувальник, який у кожен момент часу обирає для виконання задачу з найвищим пріоритетом серед готових. FreeRTOS також надає механізми синхронізації та обміну даними між задачами, що дозволяє будувати надійні та передбачувані системи реального часу.

3.3. Опишіть структуру пріоритетів задач в ОС FreeRTOS.

У FreeRTOS кожній задачі призначається числовий пріоритет під час її створення. Чим більше значення пріоритету, тим вищим є пріоритет задачі. Планувальник завжди надає процесорний час задачі з найвищим пріоритетом, яка перебуває у стані готовності. Якщо в системі присутні декілька задач з однаковим пріоритетом, то за умови увімкненого часового розділення планувальник по черзі перемикається між ними, виділяючи кожній однакові часові інтервали. Така структура дозволяє чітко визначати важливість задач і гарантувати виконання критичних операцій.

3.4. Перелічіть стани в яких можуть перебувати задачі ОС FreeRTOS.

Задачі в FreeRTOS можуть перебувати у кількох основних станах, що визначають їхню участь у плануванні. Задача може виконуватися, коли вона безпосередньо використовує процесор. Вона може бути готовою до виконання, але очікувати процесорний час через наявність задач з вищим або рівним пріоритетом. Також задача може перебувати у заблокованому стані, коли вона очікує на подію або завершення затримки. Окремо існує стан призупинення, у якому задача не бере участі в плануванні доти, доки її явно не буде відновлено.

3.5. Опишіть поняття черг в ОС FreeRTOS.

Черги в FreeRTOS є одним з основних механізмів безпечної передачі даних між задачами, а також між обробниками переривань і задачами. Черга працює за принципом першим прийшов першим вийшов і зберігає копії переданих даних, що виключає спільний доступ до однієї області пам’яті. Якщо задача намагається записати дані у заповнену чергу або зчитати дані з порожньої черги, вона може бути переведена у стан блокування до появи необхідної умови. Це дозволяє ефективно синхронізувати виконання задач та уникати станів гонитви.

3.6. Опишіть поняття семафорів в ОС FreeRTOS.

Семафори у FreeRTOS використовуються для синхронізації задач та сигналізації про настання певних подій. Вони не передають дані, а лише інформують задачі про доступність ресурсу або події. FreeRTOS підтримує декілька типів семафорів, серед яких двійкові та лічильні. Двійкові семафори часто застосовуються для синхронізації задач з перериваннями, тоді як лічильні дозволяють керувати доступом до обмеженої кількості однакових ресурсів. Семафори допомагають організувати узгоджену роботу задач без активного очікування.

3.7. Опишіть принцип доступу до захищеного ресурсу в ОС FreeRTOS.

Доступ до захищеного ресурсу в FreeRTOS організовується таким чином, щоб унеможливити одночасне використання ресурсу кількома задачами, що може призвести до помилок або пошкодження даних. Для цього використовуються критичні секції, м’ютекси або спеціальні архітектурні підходи, зокрема патерн задача-вахтер. У випадку використання задачі-вахтера лише одна задача має прямий доступ до ресурсу, а інші задачі взаємодіють з ним опосередковано через чергу. Такий підхід забезпечує безпечний та впорядкований доступ до ресурсу без складних механізмів блокування в кожній задачі.